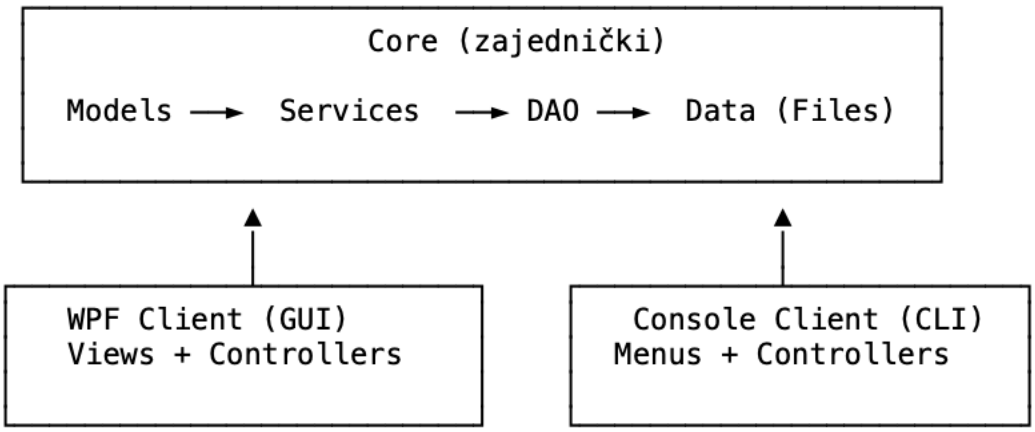
Struktura projekta: GUI + Konzola + Core

Ovaj dokument objašnjava arhitekturu aplikacije koja se sastoji od tri komponente - dva korisnička interfejsa (grafički – WPF i konzolni – CLI) i zajedničkog Core sloj koji sadrži poslovnu logiku i pristup podacima (Slika 1). Svaka komponenta se može implementirati kao poseban projekat u okviru solution-a.



Slika 1: Prikaz komponenata aplikacije

1. Core projekat – „srce aplikacije“

Core sadrži sav kod koji predstavlja poslovnu logiku, pravila aplikacije i pristup bazi. GUI i konzola ga samo koriste, ne menjaju ga.

1.1 Models (Modeli)

Modeli su klase entiteta odnosno klase koje opisuju podatke u sistemu, npr. User, Product, Order. Modeli ne sadrže logiku interfejsa, dakle treba da budu veoma jednostavne bez poslovne logike i pristupa fajl sistemu.

1.2 Services (Servisi)

Servisi sadrže poslovnu logiku. Oni koriste DAO sloj, ali ne sadrže logiku korisničkog interfejsa.

1.3 DAO (Data Access Object)

DAO je sloj za pristup podacima bilo da se oni čuvaju u bazi podataka ili u tekstualnim fajlovima kao u našem primeru. Izvršava čitanje, upis i pretragu podataka. Ne sadrži poslovnu logiku. Svaka klasa Modela, ukoliko se skladišti u fajlovima treba da ima odgovarajuću klasu DAO sloja. Imenuju se najčešće kao klasa modela sa sufiksom DAO (od klase User dobijamo UserDAO).

1.4 Interfaces

Interfejsi definišu ugovore za servise i DAO. Omogućavaju da promenimo implementaciju bez menjanja ostatka aplikacije i olakšavaju testiranje.

1.5 Utils

Pomoćne klase koje obavljaju manje zadatke (validacija, formatiranje, helperi).

2. WPF Client – Grafički korisnički interfejs (GUI)

WPF projekat sadrži Views (XAML), Controllers i App.xaml. GUI ne sadrži poslovnu logiku — samo prikazuje podatke i reaguje na događaje.

3. Console Client – Konzolni korisnički interfejs (CLI)

Console projekat omogućava korišćenje aplikacije u komandnoj liniji. Sadrži Program.cs, kontrolere i opcione komande. Koristi iste servise i modele iz Core sloja.

Zaključak

GUI i konzola pozivaju servise iz Core-a. Servisi koriste DAO za pristup bazi. Core je zajednički sloj koji obavlja sav posao, dok su GUI i CLI samo različiti načini rada aplikacije.

Ova arhitektura omogućava dobru organizaciju koda, jednostavno testiranje i jednostavno dodavanje novih interfejsa (npr. web aplikacija) bez menjanja poslovne logike. Takođe, nije obavezujuća, timovi imaju slobodu da je primene u manjoj ili većoj meri kao i da implementiraju sopstvenu organizaciju koda.

Predlog strukture projekta sa primerima klasa

```
Solution/
│
├── Core/                       (zajedničko)
│   ├── Models/
│   │   └── User.cs              // public class User { public int Id; public string Name; }
│   ├── Services/
│   │   └── UserService.cs       // public class UserService { public bool Login(string user, string pass); }
│   ├── DAO/
│   │   └── UserDao.cs           // public class UserDao : IUserDao { ... }
│   ├── Interfaces/             (common interfaces)
│   │   └── IUserDao.cs          // interface IUserDao { User GetById(int id); }
│   └── Utils/
│       └── Logger.cs            // public static class Logger { public static void Log(string msg); }
│
├── WpfClient/                  (GUI - WPF)
│   ├── Views/
│   │   └── MainWindow.xaml      // XAML UI layout
│   ├── Controllers/
│   │   └── MainController.cs
│   ├── App.xaml
│   ├── MainWindow.xaml
│   └── Program.cs
│
└── ConsoleClient/              (CLI - konzola)
    ├── Controllers/
    │   └── ConsoleController.cs // public class ConsoleController { public void Start(); }
    ├── Program.cs
    └── Commands/
        └── LoginCommand.cs       // public class LoginCommand { public void Execute(); }
```